

Taller 1 — Pilas y Colas

Documentación de diagramas UML (clases, casos de uso y actividades)



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**
Acreditación Institucional de Alta Calidad

Samuel Antonio Sanchez - 20212020151

Daniel Alejandro Chavez - 20212020109

Proyecto Curricular: Ingeniería de Sistemas

CIENCIAS DE LA COMPUTACIÓN III

HELIO HENRY RAMIREZ AREVALO.

Enlace del repositorio: <https://github.com/Donakeru/ComputerScience3-course>

Este documento describe el problema resuelto en cada ejercicio del taller y presenta, para cada uno, los tres diagramas UML que lo modelan: diagrama de clases, diagrama de casos de uso y diagrama de actividades. El código fuente completo (estructuras de datos propias Stack y Queue sobre listas enlazadas, y los algoritmos de ambos ejercicios) está disponible en el repositorio enlazado arriba.

tabla de contenido

1. Ejercicio 1 — Equilibrador de símbolos.....	3
1.1 Planteamiento del problema.....	4
1.2 Diagrama de clases.....	4
1.4 Diagrama de actividades.....	5
2. Ejercicio 2 — Asignación de tareas.....	7
2.1 Planteamiento del problema.....	7
2.2 Diagrama de clases.....	7
2.3 Diagrama de casos de uso.....	8
2.4 Diagrama de actividades.....	10

1. Ejercicio 1 — Equilibrador de símbolos

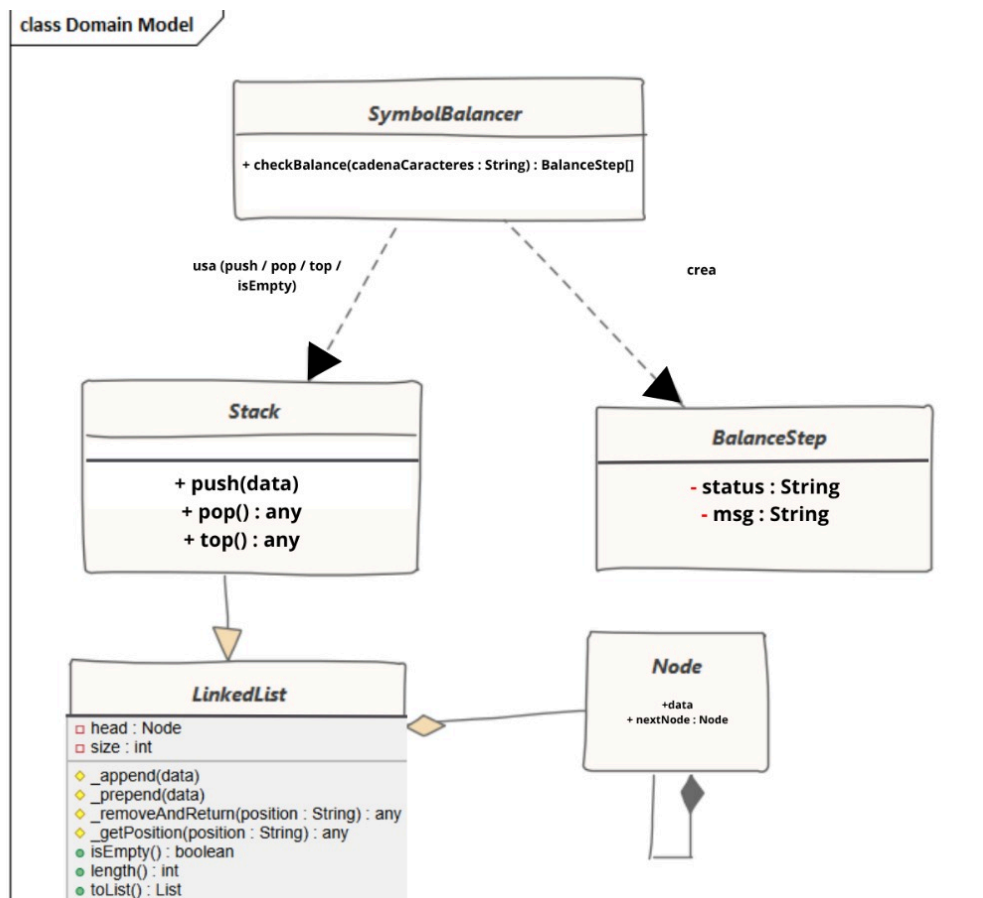
1.1 Planteamiento del problema

Dado un texto, se debe comprobar si sus símbolos de apertura y cierre — paréntesis (), corchetes [] y llaves {} — están correctamente balanceados y anidados.

El algoritmo recorre la cadena carácter a carácter usando una pila (Stack) propia, implementada sobre una lista enlazada (no arrays nativos de JavaScript): cuando encuentra un símbolo de apertura lo apila; cuando encuentra un símbolo de cierre, revisa si la pila está vacía (error: cierre sin apertura previa) o si el símbolo en el tope corresponde a ese cierre (si coincide, lo desapila; si no, error de anidación incorrecta). Al terminar de recorrer el texto, si la pila no quedó vacía, hay aperturas que nunca se cerraron. El algoritmo devuelve un resumen paso a paso que la interfaz usa para mostrar tanto el veredicto final (equilibrada / no equilibrada) como el detalle de cada paso.

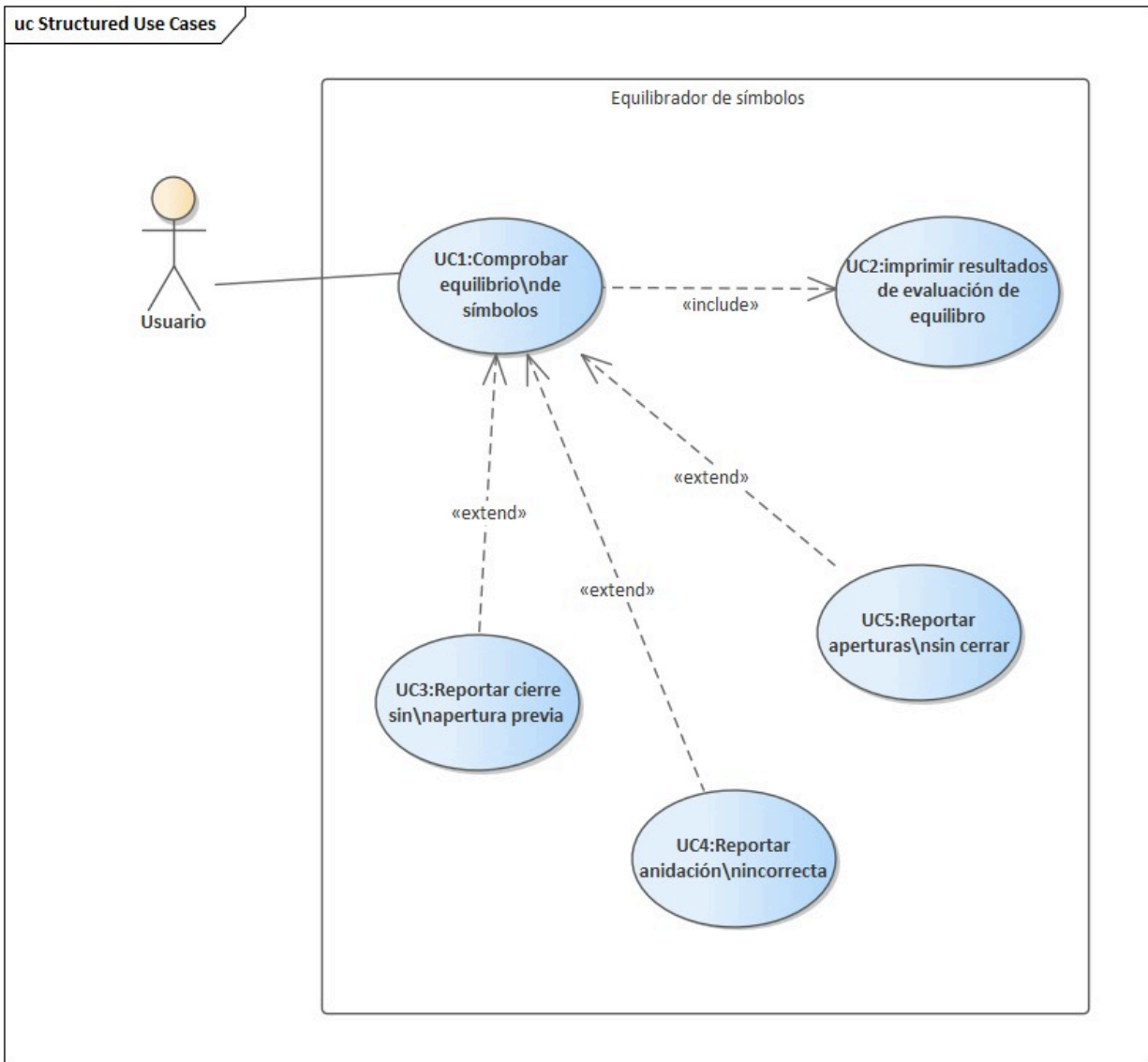
1.2 Diagrama de clases

Modela las estructuras de datos propias del proyecto (Node, LinkedList y Stack, esta última heredando de LinkedList) junto con BalanceStep (cada paso del resultado) y la clase utilitaria SymbolBalancer, que expone el algoritmo checkBalance y depende de Stack y BalanceStep.



1.3 Diagrama de casos de uso

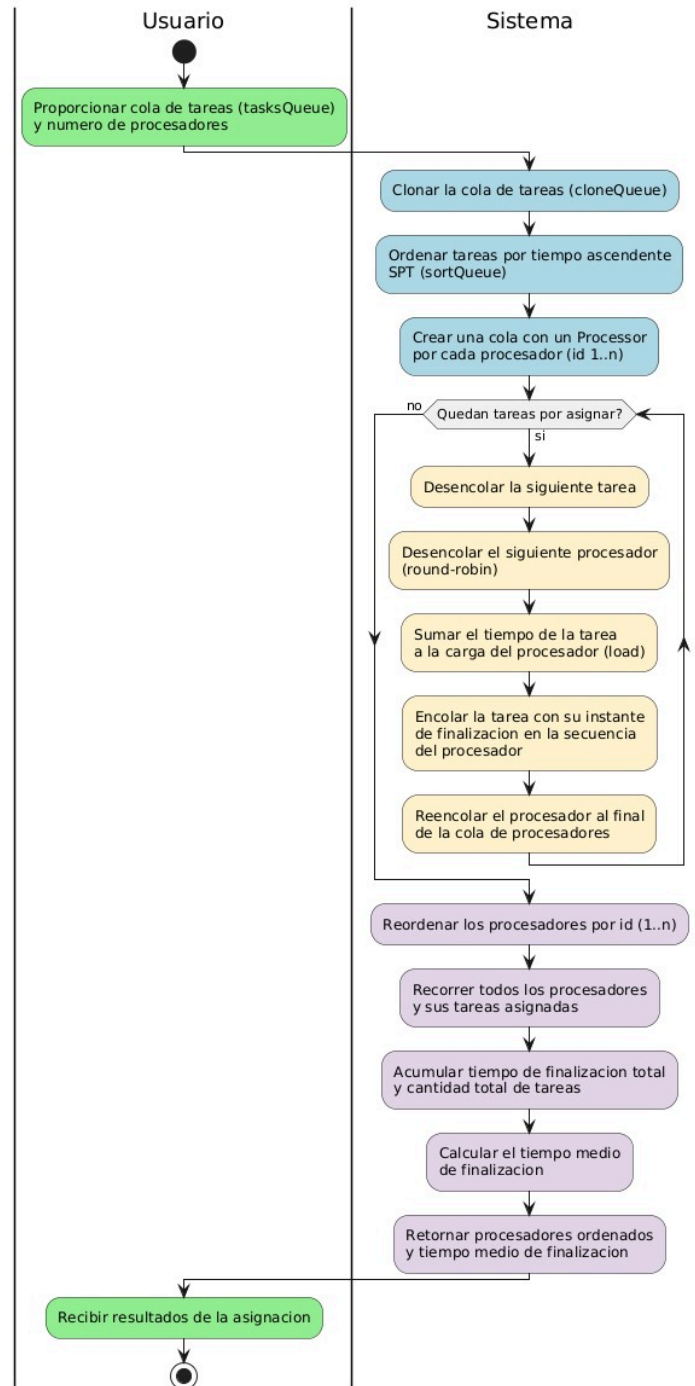
El actor Usuario se conecta únicamente al caso de uso principal "Comprobar equilibrio de símbolos", que incluye (<<include>>) el recorrido carácter a carácter, el cual a su vez incluye apilar aperturas y validar cierres. Los tres escenarios de error (cierre sin apertura, anidación incorrecta, aperturas sin cerrar) se modelan como <<extend>> porque solo se activan bajo una condición específica, no siempre que se ejecuta el caso base.



1.4 Diagrama de actividades

Representa el flujo del algoritmo checkBalance: recorrido carácter a carácter con las decisiones de apertura/cierre, la verificación del tope de la pila, y los tres puntos de salida por error, hasta el retorno del resumen final.

Ejercicio 2 - Asignacion de tareas
Diagrama de actividades (assignTasks)



2. Ejercicio 2 — Asignación de tareas

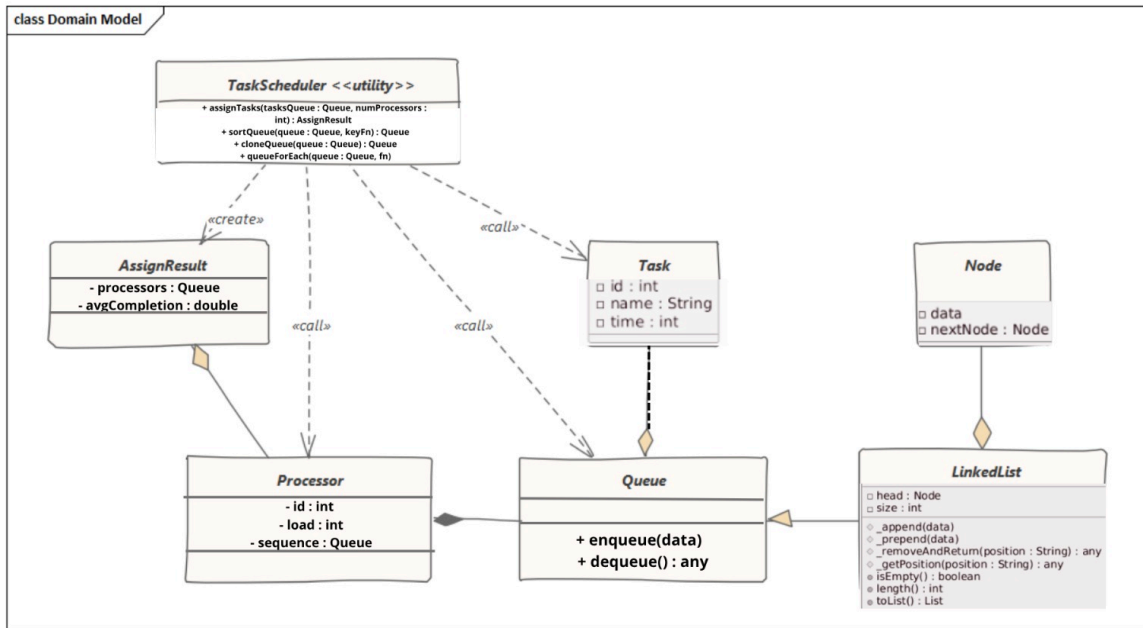
2.1 Planteamiento del problema

Dadas n tareas, cada una con un tiempo de ejecución t_i , y varios procesadores, se debe planificar el orden de ejecución que minimiza el tiempo medio de finalización (el tiempo transcurrido desde el instante 0 hasta que termina cada tarea).

El algoritmo `assignTasks` combina dos estrategias: primero ordena las tareas de menor a mayor tiempo de ejecución (SPT, Shortest Processing Time first), porque ejecutar primero las tareas cortas reduce el tiempo medio de espera; luego reparte esas tareas ya ordenadas entre los procesadores en round-robin (procesador 1, 2, ..., n , 1, ...), acumulando en cada uno su carga y el instante de finalización de cada tarea asignada. Tanto la cola de tareas pendientes como la secuencia de cada procesador se manejan como `Queue` (estructura propia, no arrays), y el ordenamiento SPT (`sortQueue`) es un selection sort implementado directamente sobre la cola.

2.2 Diagrama de clases

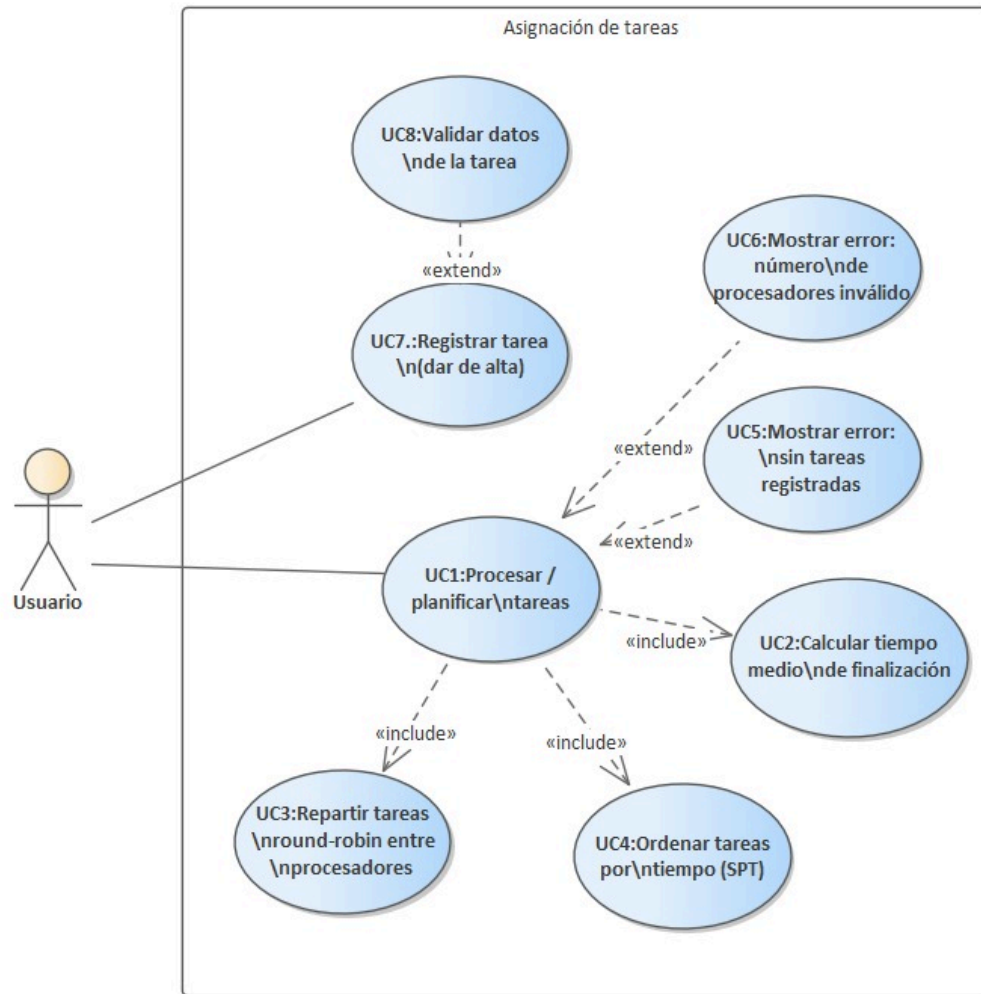
Modela `Node`, `LinkedList` y `Queue` (heredando de `LinkedList`), junto con `Task`, `Processor` (que contiene su propia `Queue` de secuencia) y `AssignResult` (el resultado de la planificación). La clase utilitaria `TaskScheduler` agrupa `assignTasks`, `sortQueue`, `cloneQueue` y `queueForEach`.



2.3 Diagrama de casos de uso

El actor Usuario se conecta a "Registrar tarea" y a "Procesar / planificar tareas". Este último incluye (<<include>>) los tres pasos que siempre ocurren al procesar: ordenar por tiempo (SPT), repartir round-robin entre procesadores y calcular el tiempo medio de finalización. La validación de datos de la tarea y los dos mensajes de error (sin tareas registradas, número de procesadores inválido) se modelan como <<extend>>, ya que solo aparecen bajo una condición puntual.

uc Structured Use Cases



2.4 Diagrama de actividades

Representa el flujo de assignTasks: clonar y ordenar la cola de tareas (SPT), crear la cola de procesadores, el ciclo de asignación round-robin (desencolar tarea, desencolar procesador, sumar carga, reencolar), y finalmente el recorrido de cierre que calcula el tiempo medio de finalización.

